USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

The Launcher, and Four Defects It Found by Existing

UM-NATOS-021

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-021	1.3 · 2026-08-15	**Complete, verified on hardware** — §6.6 corrected: the cause WAS found, and it was not the layout	Internal

1. Abstract

The first thing in this project that exists for a user rather than for a test. Everything before it was verified by reading serial output; this is verified by someone touching the glass and a program starting.

It is about 250 lines. Its value turned out to be less in what it does than in what it **demanded** — it is the first consumer to care where across the screen a finger is, the first to run the display at a rate that mattered, and the first to need a task slot. Each of those exposed a defect that had been latent for weeks:

exposed	defect	recorded in
horizontal position	touch X axis inverted since the driver was written	UM-NATOS-017 §7.1
position validity	PENIRQ answers presence, not validity; rail samples	UM-NATOS-017 §4.1
a task slot	nine `task_create` calls against a `TASK_MAX` of 8 — no idle task	§5 below
display throughput	applications and renderer blocking each other on the draw lock	UM-NATOS-014 §10

None was found by inspection. All four were found by building something that depended on them.

2. A launcher, not a window manager

Applications still render into the fixed horizontal strips `app.c` assigns by slot index. There is no focus model, no z-order, and no way for a program to own the screen and hand it back.

That boundary is deliberate rather than unfinished. Dynamic viewports need an arbitration policy for who owns which pixels, and building one before a user interface exists to demand it is designing against a guess — the same reason focus arbitration was deferred while viewports could not overlap. A launcher is the consumer that makes the question concrete; it is not yet the consumer that answers it.

2.1 Layout

```
 0   +----------+----------+----------+
     | shell    | squares  | draw     |   launcher: 3 x 3 grid
     +----------+----------+----------+   cells 80 x 70
     | paint    | notes    | ping     |
     +----------+----------+----------+
     | pong     | rogue    | 3D view  |
     +----------+----------+----------+
154  | started squares                |   status strip, expires ~2 s
168  +------------------+------+-----+
     | application 0    | name |  X  |   four strips, pitch 28, height 26
     +------------------+------+-----+   the right 60 px are the KERNEL's
     | application 1    | name |  X  |
     ...
288  +--------------------------------+
     | colour strip                   |
320  +--------------------------------+
```

Everything left of the name belongs to the application. The name and the X do not — see §6.2.

The 3D view is an icon rather than a permanent fixture: it becomes something you open, returning via the top-left corner.

2.2 Launch by name, never by index

`shell_launch(name)` looks the program up in the shell's table. The launcher holds no image pointers of its own.

This is not fastidiousness. An earlier launcher in this project indexed the table directly and silently started a *different* program than the one it named, because the table had been reordered — a `rogue` icon running `gfxrogue`. A name lookup cannot drift out of step with the table it reads.

2.3 The launcher costs nothing when idle

It repaints only when something changed, so an idle desktop pushes **zero** pixels — unlike the raycaster, which repaints unconditionally because every frame differs. If the screen looks frozen, that is correct: it is waiting for input.

3. Why a cursor on a touchscreen

A cursor is an indirection — you can already point at what you want. It earns its place here for two measured reasons: the panel is resistive and noisy, and an icon large enough to hit reliably by direct tap would leave room for very few.

The interaction is **hybrid, not modal**: a single tap MOVES the cursor to it, a double-tap OPENS what it is on. A confident user taps twice and it opens; an unsure one taps, sees where the cursor landed, and corrects. Neither has to be told which mode they are in.

4. Two defects of its own

4.1 Status text drawn over a control

The launch message was drawn at `DESK_H - 10`, which is inside the bottom-left cell's label. So "started" appeared over `pong`'s name — and `g_msg_sel` was never cleared, so it stayed there permanently.

It was reported as "*it selected Pong, which was a bit off*", and that reading was entirely reasonable. The interface was asserting something false about its own state, indefinitely.

Text that overlaps a control is not a cosmetic problem. The fix gives status its own strip below the grid, shrinks the cells to make room, excludes the strip from hit-testing, names what launched rather than saying "started", and expires after ~2 s — because a status line that never clears stops describing the present and becomes decoration.

4.2 The selection was read at the worst possible moment

Selection followed *every* sample while the finger was down, so the **last** sample before release decided it — and release is precisely when a resistive panel produces its worst data. Measured, one tap on the top-right icon:

```
first sample -> cell 2      (correct)
last sample  -> cell 3      (wrong)
```

The correct target was read, recorded, and discarded microseconds later.

Selection is now latched from the **first** sample of a press and not moved again until the finger lifts. Later samples describe a finger being *lifted*, which is not an instruction. Drag no longer moves the cursor; tap again to correct.

This was diagnosed by an instrument built for the theory and then nearly deleted when the theory looked wrong. The first/last cell pair is still reported, with a note saying why: removing an instrument once it has found its bug is how the bug returns unnoticed.

5. There was no idle task

`kmain` makes **nine** `task_create` calls; `TASK_MAX` was **8**. The ninth is the idle task, so it failed: `task_create()` returns −1 when the table is full, `task_set_idle(-1)` bounds-checks and quietly does nothing, and the return value was never printed or checked.

Two real costs. `WAITI` never executed, so the CPU never slept. And with every task asleep the scheduler found nothing runnable and fell through to "resume the interrupted context" — which resumes a **sleeping** task, the exact invariant blocking exists to enforce.

The evidence had been printed and read. The `stacks` table added hours earlier listed ids 0–7 with no idle task; it was looked at and not registered. That is the failure UM-NATOS-019 is entirely about, committed while writing the report about it.

`TASK_MAX` is now 12 and `must_create()` panics rather than returning −1, so a full table cannot fail quietly again.

6. Icons, and a close button an application cannot reach

Added in revision 1.1.

6.1 Icons are a bitmap, not nine drawing routines

8 × 8 monochrome glyphs, one byte per row, drawn at 3× into 24 × 24. Nine bespoke drawing functions would be more code than nine constants and would make every icon a place a bug can hide; a bitmap is also editable by anyone who can count to eight, which matters more than elegance for something whose only requirement is being recognisable at 24 pixels.

Each row is drawn as one rectangle per **run** of set pixels rather than one per pixel. That is not micro-optimisation: every primitive takes the draw lock, and UM-NATOS-014 §10 established that the cost of that lock is the number of acquisitions rather than the time held.

This is still not an asset pipeline (UM-NATOS-011 §6). The glyphs are in the image because there is nowhere else to put them yet.

6.2 The close button is outside the viewport, and that is the point

Every running program gets an X, and the 3D view gets one in place of the invisible top-left corner gesture it had. The two are drawn by different mechanisms and §6.5 explains why: the application buttons sit in the strips, which no full-region view touches, and the 3D view's button sits in the middle of something that repaints continuously.

The X sits in a column the application's viewport **no longer covers**: `APP_VIEW_W` is `DISP_W` minus the chrome width, and the kernel owns those pixels.

If the close button lived inside the viewport, a misbehaving program could paint over it, draw a decoy elsewhere, or simply fill its strip and hide the way out. Outside means **the one control a user needs in order to escape a program is the one control that program cannot touch.** That is the viewport argument of UM-NATOS-016 §2 applied to a pixel the *user* owns rather than one the application does.

The touch path enforces the same boundary: close buttons see the press first, and a consumed press is not passed on, so closing a program cannot also select an icon underneath it.

6.3 A button for something you cannot name

The area below the launcher was reported as "*strange space … why is there two x boxes in there*", and the answer was worse than the question implied.

Those were `ping` and `pong`, which `kmain` starts at boot to keep the IPC counters live. They exchange **messages**, not pixels. So two programs were running correctly, producing no visible output, and the only evidence of their existence was two buttons whose function was to destroy them. Tapping one would have been a guess about what it deleted.

Each running strip now carries its program's name beside its X.

The cost is real and is not hidden: the kernel column grew from 16 px to 60 px, so applications lost about a fifth of their strip width to a label they can neither draw nor overwrite. If a program later needs the full width, this is the decision to revisit.

6.4 The running marker, again

The mark for "this program is running" was a four-pixel dot in the corner of the icon. It is now an underline beneath the icon's label.

The dot had already failed once: it compared only the first character of the program name, so `paint`, `ping` and `pong` marked each other, and **nobody noticed by looking at the screen** — three wrong four-pixel dots read as a rendering artefact. It was found while writing §9 of this report.

A mark too small to be read wrongly is also too small to be read at all. The underline spans the cell.

6.5 Chrome over a repainting view has to be part of the same draw

The 3D view's close button was first drawn immediately after the view, every frame, by the same routine that draws the application buttons. It strobed badly enough that the view read as broken.

The raycaster **repaints every pixel every frame**. Anything drawn over it therefore survives only until the next frame begins, and drawing it later in the sequence cannot help — "later" ends about sixty milliseconds afterwards. The button and the frame beneath it are not two draws to be ordered. They have to be **one draw**.

`desktop_overlay_into()` writes the button into the view's framebuffer, in that buffer's own coordinates, and the raycaster calls it immediately before blitting. It is a pure function: it writes pixels and calls nothing. The frame and the button reach the panel in a single transfer, and there is no moment at which one exists without the other.

The hit test did not change. The stamped button lands on exactly the coordinates `desktop_chrome_touch()` already tested, which is worth stating because a control drawn by one mechanism and tested by another is a standing invitation for the two to drift.

With `fb off` there is no buffer to stamp into — that path blits column by column straight to the panel — so the button is drawn the old way and does flicker. An invisible exit is worse than a visible strobing one: the way out of the view would exist and be unfindable. `fb off` is a diagnostic mode rather than how it runs.

6.6 The same diagnosis, at twenty times the scope

This section is about a fix that was reverted, and it is here because the mistake was not in the reasoning.

The first attempt reached exactly the conclusion above — chrome cannot be drawn over a view that repaints continuously — and implemented it by **reserving rows** at the foot of the region for a chrome bar the views would not touch.

That is a correct solution. It also meant moving the region boundary, and therefore the application strips, and therefore the colour strip, and the launcher grid was resized to use the space freed by removing things that had been on screen at boot. Four files of constants moved together.

The result was a screen that looked wrong, and it was reverted to the last confirmed-good state without the cause being understood. Every measurement said the renderer was correct: the wall band landed at the right rows, the composed framebuffer held a dark ceiling, an orange wall and a dark floor at the expected offsets, `display_blit` was verified including its contiguous fast path, and eleven self-tests passed throughout. Three rounds of instrumenting the raycaster produced three rounds of evidence that the raycaster was fine.

The fix in §6.5 does the same job by writing 324 pixels into a buffer that was about to be sent anyway, and that comparison still stands:

> A correct diagnosis does not license a fix of arbitrary scope. The two attempts share a diagnosis and differ by a factor of twenty in what they touch.

The compile-time assertions added afterwards (§8) are the mechanical part of that lesson: the layout is agreed across four files and nothing enforced it.

6.7 The cause, found later — and it was not the layout

Revision 1.3. §6.6 said the cause was never found. It has been, and the conclusion a reader would have drawn from §6.6 was wrong.

The geometry change was applied **on its own** to current code — nothing else those commits touched — and the 3D view rendered correctly: the framebuffer allocated, frames climbed, the camera crossed the map. **Geometry alone breaks nothing.**

What broke was the camera. The relayout raised the frame rate, movement was per-frame at the time (UM-NATOS-015 §5.8), and the camera outran its turn probe and buried itself in a wall. "Blank screen" and "a bunch of vertical lines" are precisely what a wall at zero distance looks like: full-height columns of flat colour.

Which is why every instrument insisted the renderer was fine. **It was fine.** It was correctly drawing the inside of a wall. A taller region made the symptom worse rather than better, because wall height scales with `RAY_VIEW_H` — so the layout looked more guilty the more of it there was.

The actual lesson

The revert bundled the layout change **and** the camera fix into one commit. The screen looked right afterwards, which appeared to convict the layout, and the information needed to acquit it had been destroyed by the same action that produced the evidence.

> Reverting two changes together destroys the information about which one mattered. If a revert must bundle, the bundle is a hypothesis to test later, not a conclusion — and it should be recorded as one.

§6.6 recorded it as a conclusion. This section is the correction.

What is structurally true

The screen budget is real and was the other half of the difficulty. Before this change it was **312 of 320 rows allocated with 8 spare**, and the four claims on it live in four files:

rows	claimed by	file
launcher and full-region views	`DESK_H`, `RAY_VIEW_H`	`desktop.h`, `raycast.h`
application strips	`APP_VIEW_Y0/PITCH/H`	`app.h`
colour strip	`SPEC_Y/SPEC_H`	`kmain.c`

Growing one forces a cascade through the others with no single owner to check it. That is why the assertions in §8 exist, and they are what let the experiment build safely instead of silently overlapping.

The layout that resulted

```
launcher / views    0..223     was 0..167
application strips  224..287   was 168..279 — now 14 rows each
colour strip        288..319   unchanged

launcher cells      80 x 70    was 80 x 51
icon glyphs         32 x 32    was 24 x 24
notes text area     10 lines   was about 4
3D view             224 rows   was 168
```

The cost is the application strips losing more than half their height. `ping` and `pong` draw nothing, so it costs nothing today; a program that draws will notice.

7. Verification

```
tap left icon   raw_x=3408 -> x=0,   cell 0
tap right icon  raw_x=920  -> x=196, cell 2
seven consecutive taps, z 1662..2261, no rail samples
x spanning 0..187 across all three columns
first and last sample of each press agree
```

All three columns selectable, double-tap opens the named program, and the status strip reports which. Confirmed on hardware by the user.

8. Metrics

Quantity	Value
Status strip	14 px, message expires ~2 s
Double-tap window	60 ticks (~600 ms)
Minimum press	2 ticks, rejects contact chatter
SPI cost when idle	0 bytes
Grid	3 × 3, cells 80 × 70
Icon glyphs	8 × 8, drawn at 4×
Screen rows allocated, before / after	312 of 320 / 320 of 320
Layout assertions	6, spanning four files
Overlay cost per frame	324 pixels into a buffer already being sent
Reverted attempt	~200 lines, four files, cause never found
Kernel-owned column	60 px of 240 (name + close button)
Application viewport width	180 px, was 240
Defects exposed in other subsystems	4
Defects of its own	2

9. What this does not establish

- **The screen is now fully allocated.** Every row is claimed; there is no spare. The next feature wanting rows must take them from something named in §6.7, and the assertions will refuse a build that overlaps rather than letting it render wrongly.
- **Application strips are 14 rows.** No program currently draws into one, so nothing has tested whether that is enough to be useful.
- **No focus, no windows, no z-order.** §2. Applications occupy fixed strips and cannot be brought forward.
- **No way to stop a program from the launcher itself.** `kill` exists only in the shell, and the icon grid can start but not stop. Since UM-NATOS-026 the shell is *on* the grid, so `kill` no longer needs a host attached — but it is still a command typed into another app, not something the launcher does.
- **The `fb off` path still flickers.** §6.5. There is no buffer to stamp into, so the button is drawn over the panel and strobes. Accepted for a diagnostic mode; it would not be acceptable as the normal path.
- **Nothing tests the overlay.** The button's position is agreed between `desktop_overlay_into()` and `desktop_chrome_touch()` by two matching constants, and no assertion ties them together — the exact drift the layout assertions were added to prevent elsewhere.
- **The running marker is per-name, not per-instance.** Two instances of the same program would share one underline. §6.4.

- **Faulted programs cannot be dismissed.** A program that faults keeps its slot and shows no X, because it is not running — it lingers until `kill` from the shell. The new chrome makes that visible without addressing it.
- **The chrome column is fixed width.** 60 px regardless of how long the name is, and names are truncated to fit rather than scaled or scrolled.
- **Double-tap timing is unmeasured.** 600 ms and the 2-tick minimum press were reasoned, not derived from anyone's actual tapping. The counters exist to settle them and nobody has.
- **The icons are guesses at 24 pixels.** 8 × 8 glyphs scaled 3×, drawn by hand and judged by one person. `blit` and `rogue` are the two most likely to read as noise. Nothing tests whether any of them is recognisable.
- **One user.** Every judgement about whether the interaction feels right came from a single person on a single panel.

10. References

- UM-NATOS-017 §4.1, §7.1 — the touch defects this exposed
- UM-NATOS-014 §10 — the lock contention this exposed
- UM-NATOS-019 — verifying that a mechanism exists is not verifying it works
- UM-NATOS-016 §2 — the fixed viewport model the launcher does not change
- `kernel/desktop.c` — grid, cursor, press latching, status strip, `desktop_overlay_into()`
- `kernel/raycast.c` — calls the overlay immediately before its blit
- `kernel/shell.c` — `shell_launch()`, the single lookup path